UNIVERSITAS PAHLAWAN TUANKU TAMBUSAI
FAKULTAS TEKNIK — PROGRAM STUDI TEKNIK INFORMATIKA

PANDUAN PRAKTIKUM PERTEMUAN 3

Enkapsulasi: Menutup Lubang yang Kita Temukan Minggu Lalu

Mata Kuliah	Pemrograman Berorientasi Objek · 4 sks · Semester III
Sub-CPMK	Sub-CPMK-3 — Menerapkan enkapsulasi melalui atribut non-publik, property bervalidasi, dan dataclass beku, sehingga objek tidak pernah berada pada keadaan tidak valid.
Bobot terhadap nilai akhir	3%
Alokasi waktu	Terbimbing 270 menit (kuliah 100 + praktikum 170) · Penugasan terstruktur 205 menit · Belajar mandiri 200 menit
Profil gerbang	p03 — minimal dua kelas, dan minimal satu @property
Bekal dari Pertemuan 2	Repositori bertanda versi p02 berisi kelas Penyewa dan AlatTani.
Luaran pertemuan	Ketiga kelas terjaga validasinya, objek nilai Alamat yang tidak dapat diubah, lolos ketiga gerbang, bertanda versi p03.

1. MASALAH YANG KITA TINGGALKAN MINGGU LALU

Pada Pertemuan 2, Langkah 8, kita menjalankan tiga baris berikut:

```
alat._tarif_harian = -5000
print(alat.keterangan())     →  [TR-01] Traktor — Rp-5.000/hari
print(alat.biaya_sewa(3))    →  -15000
```

Python tidak protes. Gerbang tidak menangkapnya. Program terus berjalan dan menghasilkan angka minus yang masuk ke nota sewa. Pertemuan ini seluruhnya tentang menutup lubang tersebut.

2. TUJUAN DAN INDIKATOR

Setelah menyelesaikan pertemuan ini Anda mampu:

1. Membuat property baca-saja untuk data yang melekat pada objek dan tidak boleh berubah.
2. Membuat setter bervalidasi sebagai satu-satunya jalan mengubah keadaan objek.
3. Menjelaskan mengapa konstruktor wajib melewati setter, dan mengenali akibat bila tidak.
4. Membuat objek nilai yang tidak dapat diubah dengan @dataclass(frozen=True).
5. Menjelaskan bahwa garis bawah ganda bukan penjagaan sungguhan, dan membuktikannya sendiri.
6. Menulis pengujian yang membuktikan masukan salah benar-benar ditolak, bukan sekadar masukan benar diterima.

3. TIGA TINGKAT PENJAGAAN

Tidak semua data dijaga dengan cara yang sama. Pertemuan ini memperkenalkan tiga tingkat, dan keterampilan yang dinilai adalah memilih tingkat yang tepat untuk tiap data.

Tingkat	Cara	Dipakai Ketika
Baca-saja	@property tanpa setter; divalidasi di konstruktor	Data melekat pada identitas objek dan tidak masuk akal berubah. Contoh: NIK penyewa, kode alat.
Baca-tulis terjaga	@property dengan setter bervalidasi	Data boleh berubah, tetapi hanya ke nilai yang sah. Contoh: tarif harian, nomor telepon.

Tingkat	Cara	Dipakai Ketika
Beku seluruhnya	@dataclass(frozen=True)	Objek mewakili sebuah nilai, bukan sebuah benda. Diganti, bukan diubah. Contoh: alamat, periode tanggal, jumlah uang.

4. ALUR PERTEMUAN

Waktu	Kegiatan	Bentuk
0–20′	Pembukaan, pembahasan tugas kelas Petugas, dan kuis awal	Kuliah interaktif; LKM Bagian A
20–65′	Blok 1: property dan setter sebagai penjaga gerbang	Kuliah dan live coding
65–105′	Blok 2: tiga tingkat penjagaan dan objek nilai yang beku	Kuliah dan demonstrasi
105–115′	Istirahat	—
115–285′	Blok 3: praktikum terbimbing, Langkah 1 sampai 10	Praktikum; LKM Bagian B
285–305′	Blok 4: aktivitas kelompok menentukan tingkat penjagaan	Diskusi kelompok; LKM Bagian C
305–320′	Blok 5: penutup dan penugasan	Refleksi; LKM Bagian D

5. LANGKAH PRAKTIKUM

Kerjakan berurutan pada repositori yang sama. Anda akan memperbaiki kelas yang sudah ada, bukan membuat yang baru.

1
10′

Menyiapkan repositori

Aktifkan .venv, tarik pekerjaan terakhir, dan pastikan gerbang Pertemuan 2 masih hijau sebelum mengubah apa pun.

```
.venv\Scripts\activate        # Windows
source .venv/bin/activate     # Linux atau macOS
git pull

python periksa_pbo.py src --profil p02
python -m pytest
```

2
15′

Membuat property baca-saja pada AlatTani

Kode dan nama alat melekat pada identitasnya. Sekali alat terdaftar, keduanya tidak berubah. Buka src/model/alat_tani.py dan tambahkan dua property berikut.

```
    @property
    def kode(self) -> str:
        """Kode alat. Tidak dapat diubah setelah alat dibuat."""
        return self._kode

    @property
    def nama(self) -> str:
        """Nama alat. Tidak dapat diubah setelah alat dibuat."""
        return self._nama
```

Uji sendiri di terminal: buat sebuah alat, lalu coba tulis alat.kode = "TR-99". Python akan menolak dengan AttributeError, karena property tanpa setter memang tidak bisa ditulisi. Inilah tingkat penjagaan pertama, dan ia gratis.

3
25′

Membuat setter bervalidasi untuk tarif

Tarif boleh berubah — UPJA memang menyesuaikan tarif tiap musim. Yang tidak boleh adalah tarif menjadi nol, minus, atau tidak masuk akal. Ganti seluruh isi kelas dengan bentuk berikut.

```
    def __init__(self, kode: str, nama: str, tarif_harian: int) -> None:
        if not kode.strip():
            raise ValueError("kode alat tidak boleh kosong")
        self._kode = kode
        self._nama = nama
        self._tarif_harian = 0
        self.tarif_harian = tarif_harian      # ← LEWAT SETTER

    @property
    def tarif_harian(self) -> int:
        """Tarif sewa per hari dalam rupiah."""
        return self._tarif_harian

    @tarif_harian.setter
    def tarif_harian(self, nilai: int) -> None:
        if nilai <= 0:
            raise ValueError(f"tarif harus lebih dari nol, diterima {nilai}")
        if nilai > 5_000_000:
            raise ValueError(f"tarif tidak wajar, diterima {nilai}")
        self._tarif_harian = nilai
```

Perhatikan dua baris di dalam konstruktor. Baris self._tarif_harian = 0 hanya menyiapkan tempat agar mypy tahu tipenya. Baris berikutnya, self.tarif_harian = tarif_harian, tampak seperti penugasan biasa padahal ia memanggil setter. Di situlah validasinya bekerja.

4
15′

Kesalahan yang disengaja ①: konstruktor melewati penjaga

Buat berkas sementara konstruktor_bocor.py di folder utama. Perhatikan bahwa setter-nya ada, validasinya benar, dan tetap saja gagal.

```
    def __init__(self, kode: str, nama: str, tarif_harian: int) -> None:
        self._kode = kode
        self._nama = nama
        self._tarif_harian = tarif_harian   # LANGSUNG, tidak lewat setter

    @tarif_harian.setter
    def tarif_harian(self, nilai: int) -> None:
        if nilai <= 0:
            raise ValueError(f"tarif harus lebih dari nol, diterima {nilai}")
        self._tarif_harian = nilai


alat = AlatTani("TR-01", "Traktor", -5000)   # lolos tanpa keluhan
print(alat.tarif_harian)                     # -5000
print(alat.biaya_sewa(3))                    # -15000
alat.tarif_harian = -9000                    # BARU ditolak di sini
```

Penjaganya ada, pintunya dilewati. Setter menolak -9000 dengan benar, tetapi -5000 sudah masuk lebih dulu lewat konstruktor. Objek lahir dalam keadaan tidak valid, dan sisa program mempercayainya. Inilah sebabnya baris self.tarif_harian = tarif_harian pada Langkah 3 ditulis begitu, bukan self._tarif_harian = tarif_harian. Satu garis bawah membedakan program yang terjaga dari program yang hanya kelihatan terjaga. Catat pada LKM Bagian B.1.

5
20′

Membuat objek nilai Alamat

Alamat bukan benda yang berubah. Kalau seseorang pindah, ia tidak mengubah alamatnya — ia menempati alamat lain. Buat berkas src/model/alamat.py.

```
"""Model domain: alamat penyewa, objek nilai yang tidak dapat diubah."""

from dataclasses import dataclass


@dataclass(frozen=True)
class Alamat:
    """Alamat tempat tinggal penyewa. Sekali dibuat, tidak dapat diubah."""

    dusun: str
    desa: str
    kecamatan: str

    def __post_init__(self) -> None:
        if not self.desa.strip():
            raise ValueError("nama desa tidak boleh kosong")
        if not self.kecamatan.strip():
            raise ValueError("nama kecamatan tidak boleh kosong")

    def lengkap(self) -> str:
        """Mengembalikan alamat lengkap dalam satu baris."""
        return f"Dusun {self.dusun}, Desa {self.desa}, Kec. {self.kecamatan}"

    def singkat(self) -> str:
        """Mengembalikan desa dan kecamatan saja, untuk nota sewa."""
        return f"{self.desa}, {self.kecamatan}"
```

Tiga hal yang berbeda dari kelas biasa: atributnya ditulis tanpa garis bawah dan tanpa konstruktor, validasinya diletakkan pada __post_init__ yang dijalankan otomatis setelah objek terbentuk, dan mencoba mengubahnya kemudian akan melempar FrozenInstanceError.

6
15′

Memakai Alamat di dalam Penyewa

Ganti atribut desa yang bertipe teks dengan objek Alamat. Perhatikan bahwa Penyewa kini menyimpan sebuah objek, bukan sekadar sebuah kata.

```
from src.model.alamat import Alamat


class Penyewa:
    """Identitas seorang petani yang menyewa alat dari UPJA."""

    def __init__(self, nik: str, nama: str, alamat: Alamat,
                 telepon: str = "-") -> None:
        ...
        self._alamat = alamat

    @property
    def alamat(self) -> Alamat:
        """Alamat penyewa. Objeknya sendiri tidak dapat diubah isinya."""
        return self._alamat

    def identitas(self) -> str:
        return f"{self._nama} ({self._nik}) — {self._alamat.singkat()}"
```

Ini keuntungan tersembunyi objek beku: property alamat boleh mengembalikan objeknya apa adanya tanpa khawatir. Siapa pun yang menerimanya tidak akan bisa merusaknya. Kalau Alamat bukan objek beku, mengembalikannya berarti menyerahkan kendali.

7
15′

Menjaga NIK dan nomor telepon

NIK adalah identitas: dijaga di konstruktor, lalu dikunci dengan property tanpa setter. Nomor telepon boleh berubah, jadi ia mendapat setter.

```
        if len(nik) != 16 or not nik.isdigit():
            raise ValueError(f"NIK harus 16 digit angka, diterima '{nik}'")

    @property
    def nik(self) -> str:
        """NIK penyewa. Melekat seumur hidup, tidak dapat diubah."""
        return self._nik

    @telepon.setter
    def telepon(self, nilai: str) -> None:
        bersih = nilai.replace("-", "").replace(" ", "")
        if nilai == "-":
            self._telepon = "-"
            return
        if not bersih.isdigit() or len(bersih) < 10:
            raise ValueError(f"nomor telepon tidak sah, diterima '{nilai}'")
        self._telepon = nilai
```

8
15′

Kesalahan yang disengaja ②: garis bawah ganda bukan kunci

Banyak yang mengira dua garis bawah membuat atribut benar-benar privat. Jalankan potongan berikut di terminal dan buktikan sendiri.

```
python -c "
class Dompet:
    def __init__(self, saldo: int) -> None:
        self.__saldo = saldo
    @property
    def saldo(self) -> int:
        return self.__saldo

d = Dompet(100000)
try:
    print(d.__saldo)
except AttributeError as e:
    print('Akses langsung  : ditolak ->', e)
d._Dompet__saldo = -1
print('Lewat nama asli : berhasil ->', d.saldo)
print('Isi objek       :', list(vars(d)))
"
```

Keluaran:

```
Akses langsung  : ditolak -> 'Dompet' object has no attribute '__saldo'
Lewat nama asli : berhasil -> -1
Isi objek       : ['_Dompet__saldo']
```

Yang sebenarnya terjadi. Python mengganti nama __saldo menjadi _Dompet__saldo. Itu bukan penguncian, melainkan penggantian nama untuk mencegah tabrakan pada pewarisan. Siapa pun yang tahu polanya bisa menembusnya, dan vars() membocorkan nama aslinya. Kesimpulan yang harus Anda pegang: Python tidak punya privat sungguhan. Yang menjaga kode Anda adalah setter yang selalu dilewati, pengujian yang membuktikan penolakan, dan tinjauan kode — bukan jumlah garis bawah. Catat pada LKM Bagian B.2.

9
25′

Menulis pengujian yang membuktikan penolakan

Sampai Pertemuan 2 pengujian Anda hanya membuktikan masukan benar diterima. Mulai sekarang yang lebih penting adalah membuktikan masukan salah ditolak. Gunakan pytest.raises.

```
import dataclasses

import pytest

from src.model.alamat import Alamat
from src.model.alat_tani import AlatTani


def test_tarif_minus_ditolak_sejak_konstruktor() -> None:
    with pytest.raises(ValueError):
        AlatTani("TR-01", "Traktor", -5000)


def test_tarif_minus_ditolak_lewat_setter() -> None:
    alat = AlatTani("TR-01", "Traktor", 150000)
    with pytest.raises(ValueError):
        alat.tarif_harian = -5000
    assert alat.tarif_harian == 150000        # nilai lama tetap utuh


def test_kode_tidak_punya_setter() -> None:
    alat = AlatTani("TR-01", "Traktor", 150000)
    with pytest.raises(AttributeError):
        alat.kode = "TR-99"      # type: ignore[misc]


def test_alamat_tidak_dapat_diubah() -> None:
    a = Alamat("Pulau Jambu", "Kuok", "Kuok")
    with pytest.raises(dataclasses.FrozenInstanceError):
        a.desa = "Salo"          # type: ignore[misc]
```

Minimal lima belas fungsi uji seluruhnya, dan minimal delapan di antaranya membuktikan penolakan. Perhatikan baris assert setelah pytest.raises pada uji kedua: ia membuktikan bahwa penolakan tidak merusak nilai yang sudah ada. Komentar type: ignore diperlukan karena mypy tahu penugasan itu memang salah — dan kita memang sengaja mengujinya.

10
15′

Menjalankan ketiga gerbang dan mengumpulkan

```
python periksa_pbo.py src --profil p03
python -m mypy
python -m pytest
python -m src.main
```

Keluaran yang diharapkan:

```
LOLOS. Tidak ada pelanggaran disiplin berorientasi objek.
Success: no issues found in 10 source files
15 passed

Budi Santoso (1406012509900001) — Kuok, Kuok
Dusun Pulau Jambu, Desa Kuok, Kec. Kuok
[TR-01] Traktor Roda Dua Kubota — Rp150.000/hari

--- Percobaan menembus penjagaan ---
Ditolak (tarif nol): tarif harus lebih dari nol, diterima 0
Ditolak (tarif minus): tarif harus lebih dari nol, diterima -5000
Tarif tetap utuh: [TR-01] Traktor Roda Dua Kubota — Rp150.000/hari
```

Hapus konstruktor_bocor.py, lalu simpan pekerjaan Anda.

```
git add .
git commit -m "P03: enkapsulasi tarif, NIK, telepon, dan objek nilai Alamat"
git tag p03
git push origin main --tags
```

6. AKTIVITAS KELOMPOK: MENENTUKAN TINGKAT PENJAGAAN

Berkelompok, 20 menit. Kembali ke kelas Keramba yang kelompok Anda rancang pada Pertemuan 2.

Untuk setiap data di bawah ini, tentukan tingkat penjagaannya — baca-saja, baca-tulis terjaga, atau bagian dari objek beku — dan tuliskan alasannya. Isi pada LKM Bagian C.

Data	Pertanyaan Penuntun	Petunjuk
kode keramba	Apakah masuk akal berubah setelah keramba terdaftar?	—
luas dalam meter persegi	Apakah ukuran fisik keramba berubah sendiri?	—
jumlah bibit ditebar	Apakah pembudidaya bisa menambah tebaran di tengah siklus?	—
lokasi keramba	Apakah ini sebuah benda, atau sebuah nilai?	Bandingkan dengan Alamat
nama pemilik	Apakah keramba bisa berpindah tangan?	—
tanggal tebar	Apakah peristiwa yang sudah terjadi bisa diubah?	—

Satu butir tambahan yang bernilai paling tinggi: temukan satu data yang menurut kelompok Anda tidak layak dijaga sama sekali — artinya boleh dibiarkan terbuka — dan jelaskan alasannya. Kelompok yang menjawab "semuanya harus dijaga" kehilangan nilai butir ini.

7. PENUGASAN TERSTRUKTUR

Tugas	Perbaiki kelas Petugas yang Anda buat pada tugas Pertemuan 2 agar seluruh atributnya terjaga sesuai tingkat yang tepat.
Ketentuan	Minimal satu property baca-saja, minimal satu setter bervalidasi, dan minimal satu objek nilai beku yang Anda rancang sendiri (misalnya PeriodeTugas atau NomorPegawai).
Pengujian	Minimal enam fungsi uji, dan minimal empat di antaranya membuktikan penolakan menggunakan pytest.raises.
Yang dinilai khusus	Alasan pemilihan tingkat penjagaan. Sertakan komentar satu baris di atas setiap property yang menjelaskan mengapa tingkat itu yang dipilih.
Gerbang	Wajib lolos periksa_pbo.py --profil p03, mypy, dan pytest. Tanda versi p03-tugas.
Tenggat	Pukul 23.59 pada hari sebelum Pertemuan 4.
Belajar mandiri	Rujukan [R1] Bab 3, [R2] Bab 11 dan 22, serta [R3] Item 44–47.

8. DAFTAR PERIKSA SEBELUM MENINGGALKAN LABORATORIUM

☐	AlatTani memiliki property baca-saja untuk kode dan nama.
☐	Konstruktor AlatTani menugaskan tarif lewat setter, bukan langsung ke atribut.
☐	Kelas Alamat sudah ada, memakai @dataclass(frozen=True) dan __post_init__.
☐	Penyewa menyimpan objek Alamat, bukan lagi teks desa.
☐	NIK divalidasi di konstruktor dan tidak punya setter.
☐	Pengamatan konstruktor bocor sudah dicatat pada LKM Bagian B.1.
☐	Pembongkaran garis bawah ganda sudah dicatat pada LKM Bagian B.2.
☐	Minimal lima belas fungsi uji, minimal delapan membuktikan penolakan.
☐	Ketiga gerbang hijau dengan profil p03; berkas konstruktor_bocor.py sudah dihapus.
☐	Tanda versi p03 sudah terdorong ke GitHub.